

FINIO

AI-Powered Personal Finance Analyser

Master Build Plan v2 — Web App + Database + B2B API

Built by Ammar Hannun

Deployed on GitHub → Railway + Vercel + Supabase

"Finio tells you not just where your money went — but exactly where it should go next."

1. Project Vision

Finio is an AI-powered personal finance analyser built specifically for young Australians (18–30). Users export their CommBank CSV statement and Finio does the rest — cleaning messy merchant data, classifying transactions using a real ML model, detecting spending habits, forecasting savings goals, and delivering personalised financial coaching through Claude AI.

Unlike every existing Australian budgeting app, Finio tells you what to do with your money — not just what you spent. It combines machine learning, an LLM-powered coach, and real mathematical formulas to give users actionable, personalised insight. It is also built as a B2B API product from day one, so Australian banks and fintechs can integrate Finio intelligence into their own platforms.

Why Finio wins

Competitor	What Finio does better
CommBank app	CommBank shows transactions. Finio explains them, predicts them, and tells you what to do next.
ChatGPT	ChatGPT gives generic advice. Finio uses your actual data, does real math, tracks improvement over time.
WeMoney	WeMoney categorises manually. Finio uses a trained ML classifier, AI coach, and spend verdict engine.
Pocketbook (dead)	Shut down in 2022. 16M CommBank users need an alternative. Finio fills that gap.

Any existing app	None offer: ML pattern detection + AI coach + Spend Check + investing nudges + B2B API.
------------------	---

2. App Navigation — 5 Pages

The web app has 5 pages accessible via a top navigation bar. Each page has a focused job.

Page	Job	Key features
Dashboard	Your money at a glance	CSV upload, 4 metric cards, streak badge, charts, budget bars, personality card
Patterns	Habits you didn't notice	Pattern cards, severity badges, bill detector, weekend vs weekday, day-of-week chart
AI Coach	Your personal finance advisor	Claude chat with full financial context injected, quick questions, conversation history
Spend Check	Can I afford this right now?	Item + amount input, green/yellow/red verdict, safety buffer breakdown, invest vs spend
Invest	What to do with what's left	50/30/20 split, ETF nudges, savings goal tracker, progress tracker, First \$1,000 mode

3. Full Feature List

Consumer Features

1. Dashboard

- 4 metric cards: Total Spent, Income, Daily Burn Rate, Risk Score 0–100
- Streak badge — consecutive weeks under budget (e.g. '🔥 3 week streak')
- Donut chart (spending by category), line chart (balance over time), bar chart (top merchants)
- Budget progress bars per category — green under limit, red over limit
- Financial Personality card — your type + 3-step AI action plan

2. Auto-Categorisation (ML Text Classifier)

- Reads messy CommBank merchant names (e.g. 'UBER* EATS AU 12345 SYDNEY')
- Classifies into: Food Delivery, Groceries, Transport, Subscriptions, Coffee, Entertainment, Health, Other
- Built with Naive Bayes text classifier — trained on labelled Australian merchant data
- User can correct any category — model retrains on corrections

3. Spending Patterns

- Detects: food delivery habit, subscription overload, weekend overspending, coffee latte effect, end-of-month risk, payday spike
- Each pattern shows severity (High/Medium/Low), dollar impact, and a recommendation
- Charts: weekend vs weekday, day-of-week breakdown, monthly spending trend

4. AI Coach — Claude API

- Powered by Claude (claude-haiku-3-5) — fast and cost-effective
- User's full financial summary injected as context before every conversation
- Free text chat + 6 quick-question buttons
- Conversation history saved in Supabase per user
- Rule-based fallback if no API key — app still works without it

5. Spend Check

- Enter item name + amount → instant green/yellow/red verdict
- Formula: projected balance – (daily burn × days until payday) = safety buffer
- Green ≤30% of buffer, Yellow ≤60%, Red >60%
- 'Invest vs Spend' comparison — shows 10-year compound value of investing the amount instead

6. Budget Setter — AI-Generated

- Automatically suggests realistic budgets based on 2–3 months of spending history
- Claude explains each suggestion in plain English — 'We suggest \$280 for food delivery because you averaged \$340 and cutting 20% is realistic'
- User can edit any category or regenerate all suggestions
- Progress bars fill as the user spends — stored in Supabase

7. Savings Tracker + Forecaster

- User sets a goal: 'Save \$1,000 by August'
- Linear regression model forecasts whether they will hit the goal based on current trajectory
- Shows: on track / off track, weekly savings required, projected end-of-month balance

8. Bill Detector

- DBSCAN clustering detects recurring payments (rent, Netflix, gym, phone bill)
- Shows amount, frequency, and next predicted due date for each bill
- Alerts user when a bill is due within 3 days

9. 50/30/20 Engine

- Splits income automatically: 50% needs, 30% wants, 20% savings+investing
- Compares actual spending vs ideal split in a visual chart
- Tells user exactly how far off each bucket is in dollars

10. Financial Personality Score

- Assigns a type based on spending patterns: Impulse Buyer, Subscription Hoarder, Weekend Warrior, Sensible Saver
- Claude writes a personalised 3-step action plan for each type
- Displayed as a card on the Dashboard

11. Investment Nudges

- Shows how much is left after the 50/30/20 split
- Recommends where to put it: ING/Macquarie savings (~5.35% p.a.), VGS/A200 ETFs, voluntary super
- Claude explains each recommendation in plain language matched to the user's age and income
- Disclaimer: 'General information only — not financial advice'

12. Progress Tracker

- Upload next month's CSV and see exact improvement vs last month per category
- Green arrows (improved), red arrows (worse), percentage change
- Trend line across all uploaded months stored in Supabase

13. Streak System

- Tracks consecutive weeks where user stayed under their total budget
- Displayed as a badge: '🔒 3 week streak' on the Dashboard
- Resets if the user goes over budget in any week — stored in Supabase

14. First \$1,000 Mode

- Simplified mode on the Invest page for users who have never saved before
- Single focus: hit the first \$1,000 milestone
- Gamified progress bar with encouraging milestone messages

B2B Features — API Product

- Clean REST API — any bank or fintech sends transaction data, gets back insights
- API key authentication + rate limiting
- Full Swagger/OpenAPI documentation auto-generated by FastAPI
- Webhook support for real-time alerts (bill due, over budget, goal hit)
- MCP Server — banks integrate Finio tools into their own AI assistants

4. AI Integration — Full Picture

Claude API is used in 4 places. ML models handle 3 more. Algorithms cover the rest. The app works without any API key — rule-based fallbacks are built for everything.

Where	What Claude / AI does	Type
AI Coach	Full financial summary injected as context. Answers questions using real data, not generic advice.	Claude API (LLM)
Budget Setter	Analyses category averages, explains each budget suggestion in plain English, responds to edits.	Claude API (LLM)
Personality Score	Writes a personalised 3-step action plan for your spending personality type.	Claude API (LLM)
Invest Nudges	Explains each ETF/savings recommendation in plain language based on user's age and income bracket.	Claude API (LLM)
Auto-categorise	Naive Bayes text classifier trained on Australian merchant names. Fast, accurate, retrainable.	ML model (sklearn)

Bill Detector	DBSCAN clustering finds recurring payments by amount + interval. No manual input needed.	ML model (DBSCAN)
Savings Forecast	Linear regression on balance history predicts whether user hits savings goal by target date.	ML model (regression)
Spend Check	Custom formula: projected balance – (burn rate × days until payday) = safety buffer → verdict.	Custom algorithm
50/30/20 Engine	Statistical split of income into needs/wants/savings. Compares actuals to ideal. Flags the gap.	Statistical algorithm

5. Tech Stack — Updated

Layer	Technology	Purpose	Cost
Backend	Python + FastAPI	All logic + REST API	Free
ML Models	scikit-learn	Classifier, clustering, regression	Free
AI Coach	Claude API (Haiku)	LLM conversations	Pay per use (~\$0.01/chat)
Data	pandas + numpy	CSV parsing + analysis	Free
Database	Supabase (PostgreSQL)	All user data, multi-user	Free tier
Auth	Supabase Auth	Login, signup, sessions	Free (included)
Frontend	Claude-generated HTML/JS	Web UI	Free
Code hosting	GitHub	Version control + CI/CD	Free
Backend deploy	Railway	Auto-deploy from GitHub	Free tier
Frontend deploy	Vercel	Auto-deploy from GitHub	Free
Domain	Skip for now	finio.railway.app works fine	\$0
TOTAL COST	—	—	~\$0 to launch

6. Database Schema — Supabase

All data is stored in Supabase (PostgreSQL). Each table is tied to a user ID. This enables multi-user support from day one.

Table	Key columns	Purpose
users	id, email, age, income_bracket, created_at	One row per account
transactions	user_id, date, amount, merchant, category, is_expense	Every uploaded transaction
budgets	user_id, category, monthly_limit, month	Budget per category per month
snapshots	user_id, month, summary_json	Monthly summary for progress tracker
goals	user_id, name, target_amount, target_date, current_saved	Savings goals
streaks	user_id, current_streak, best_streak, last_updated	Streak tracking
chat_history	user_id, role, message, timestamp	AI Coach conversation history

api_keys	user_id, key_hash, tier, created_at	B2B API access keys
----------	--	---------------------

7. Folder Structure

```
finio/ ├── backend/ |   ├── main.py           ← FastAPI app + all routes |
|── config.py         ← colours, keywords, constants |   ├── requirements.txt
|   ├── .env          ← Claude API key + Supabase keys |   └── modules/ |
|── commbank_parser.py ← reads + cleans CommBank CSV |   ├── data_processor.py
← adds flags + summary metrics |   ├── categoriser.py      ← ML text classifier
(sklearn) |   ├── analytics.py          ← patterns + risk score |   ├──
spend_check.py         ← verdict formula |   ├── budget_setter.py      ← AI budget
suggestions (Claude) |   ├── savings_forecaster.py ← linear regression model |
|── bill_detector.py    ← DBSCAN clustering |   ├── ai_coach.py         ←
Claude API coach |   ├── history.py      ← month snapshots + streaks |
|── personality.py      ← financial personality score |   └── invest.py
← 50/30/20 + ETF nudges |── frontend/      ← Claude-generated web UI
|── data/ |   └── sample_transactions.csv ← fake CommBank data for testing |
|   ├── .github/ |   ├── workflows/ |   └── deploy.yml      ← auto-deploy on push
```

8. Deployment Flow

No manual deployment. Every push to GitHub triggers automatic deployment to Railway and Vercel.

Cursor (write code) → GitHub (push) → Railway (backend live) + Vercel (frontend live)

Platform	URL	What it hosts	Cost
Railway	finio-backend.railway.app	Python backend	Free
Vercel	finio.vercel.app	Web frontend	Free
Supabase	supabase.com dashboard	DB + auth	Free
TOTAL LAUNCH COST	\$0 — all free tiers		\$0

9. Step-by-Step Build Plan

Phase	What you build	Files / tools	Time
1 — Setup	Project structure + environment	Cursor, GitHub repo, venv, requirements.txt, .env, sample CSV	1 day
2 — Auth + DB	User accounts + database	Supabase project, tables, auth, Python client	2 days
3 — Data layer	Read + clean CommBank CSV	commbank_parser.py, data_processor.py	3–4 days
4 — ML models	Train all 3 models	categoriser.py, bill_detector.py, savings_forecaster.py	1 week
5 — Core logic	All analysis + verdicts	analytics.py, spend_check.py, budget_setter.py, history.py, personality.py, invest.py	1 week
6 — AI layer	Claude API integration	ai_coach.py, budget AI, personality AI, invest nudges	3–4 days
7 — API layer	FastAPI + B2B + MCP	main.py, all endpoints, auth, rate limiting, Swagger, MCP server	3–4 days
8 — Frontend	Full web UI	Claude generates HTML/JS, connect to API, test all pages	2–3 days
9 — Deploy	Push live to GitHub	Railway + Vercel auto-deploy, Supabase live, smoke test	2 days

TOTAL	Live, multi-user, deployed	Portfolio-ready + shareable link on resume	~5–6 wks
-------	----------------------------	--	----------

--	--	--	--

10. Resume Skills This Project Covers

Skill	Where in Finio
NLP / Text Classification	categoriser.py — Naive Bayes on Australian merchant names
Unsupervised ML / Clustering	bill_detector.py — DBSCAN detects recurring bills
Predictive ML / Regression	savings_forecaster.py — linear regression on balance trajectory
Feature Engineering	data_processor.py — ML-ready flags from raw bank CSV
Model Evaluation	Accuracy, precision, recall for text classifier
LLM Integration	ai_coach.py — Claude API with context injection
Prompt Engineering	Financial summaries structured as LLM system prompts
REST API Design	main.py — FastAPI with auth, rate limiting, Swagger
Database Design	Supabase schema — multi-user, relational, normalised
Authentication	Supabase Auth — email/password + session management
Data Engineering	Parsing, cleaning, transforming real bank data with pandas
Statistical Analysis	50/30/20 engine, burn rate, pattern detection
Algorithm Design	Spend Check formula, budget suggestion rules, streak logic
MCP Development	Finio MCP server for bank AI assistant integration
Full-Stack Deployment	GitHub → Railway + Vercel + Supabase CI/CD pipeline
Product Design	User research, competitive analysis, real problem solved

11. B2B API Strategy

Finio's backend is a clean API from day one. Any Australian bank, neobank, or fintech can integrate Finio's intelligence layer into their own platform and pay per API call.

Target partners

- Up Bank — neobank, tech-forward, likely early adopter
- Frollo — already in the open banking space
- WeMoney — could replace their basic analytics with Finio's ML engine
- Any fintech startup needing a ready-made financial intelligence API

REST endpoints

Method	Endpoint	What it does
--------	----------	--------------

POST	/upload	Send CSV → get full analysis, categories, metrics
GET	/patterns	Detected spending habits with severity + dollar impact
GET	/budget	AI-suggested budgets per category (Claude-generated)
POST	/spendcheck	Send item + amount → get green/yellow/red verdict
POST	/coach	Send message → get Claude response with financial context
GET	/progress	Month-on-month improvement comparison
GET	/bills	Detected recurring payments + next due dates
GET	/invest	50/30/20 split + ETF recommendations

Start with Phase 1. Build one file at a time. Ship it.

This is a real product. 14 features. Real ML. Real AI. Real users. Build it like one.